MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers Informatics and Microelectronics
Department of Software Engineering and Automation

Student:
Tacu Igor
FAF-242

REPORT

Laboratory Works No 7 and No 8

Three Small Programs Processes Redirection and a Pipe

The Same Calls Inside Your Own OS

Operating Systems

Checked by:
Universitary Assistant
Reițman Patricia

UTM
Chișinău 2026

Purpose of the Laboratory Works

Laboratory Work No. 7 investigates process creation, program replacement, process waiting, zombie states, file-descriptor redirection, and pipelines on Linux. Laboratory Work No. 8 repeats the process-control operations inside xv6 and implements bidirectional process communication with two pipes. Together, the works connect familiar shell behavior to the system calls and descriptor rules that implement it.

Objectives

1. Create a child with fork, replace it with exec, and collect its termination status with wait or waitpid.

2. Observe a zombie process and explain why waiting removes it from the process table.

3. Redirect standard output by closing descriptor 1 and reopening a file in its place.

4. Construct ls /etc | wc -l with a pipe and two children, then explain pipe EOF.

5. Register and execute new user programs inside xv6.

6. Exchange a byte between parent and child with separate parent-to-child and child-to-parent pipes.

Experimental Environment

The Linux experiments were performed on a remote Ubuntu 24.04.3 LTS system with GCC 13.3.0. The Linux programs were executed from ~/os-lab7. The xv6-riscv tree was compiled with the RISC-V GNU toolchain and executed under qemu-system-riscv64. Lab 7 commands were run in the remote root session, while the xv6 work was performed as user loredana.

Report Structure

Part I presents the four Linux experiments from Laboratory Work No. 7. Part II presents the xv6 run and pingpong programs from Laboratory Work No. 8. The appendices contain the complete source code, followed by a joint conclusion and critical analysis.

1 Laboratory Work No 7 Three Small Programs

1.1 Process Creation Program Replacement and Waiting

The program printed its parent PID and called fork. The child displayed its PID and parent PID, then replaced itself with ls -l using execlp. The parent waited for the child and printed its exit status. The source files were compiled with GCC.

```
gcc -std=c11 -Wall -Wextra -o run run.c
gcc -std=c11 -Wall -Wextra -o zombie zombie.c
gcc -std=c11 -Wall -Wextra -o redirect redirect.c
gcc -std=c11 -Wall -Wextra -o pipeline pipeline.c
./run
```

```
root@ubuntu-4gb-hel1-1:/# cd ~/os-lab7
root@ubuntu-4gb-hel1-1:~/os-lab7# pwd
/root/os-lab7
root@ubuntu-4gb-hel1-1:~/os-lab7# ls -l run.c zombie.c redirect.c pipeline.c
-rw-r--r-- 1 root root 753 Oct 19 12:30 pipeline.c
-rw-r--r-- 1 root root 622 Oct 19 12:30 redirect.c
-rw-r--r-- 1 root root 710 Oct 19 12:29 run.c
-rw-r--r-- 1 root root 725 Oct 19 12:29 zombie.c
root@ubuntu-4gb-hel1-1:~/os-lab7# grep -n 'execlp\|sleep(20)' run.c zombie.c
run.c:23:        execlp("ls", "ls", "-l", (char *)NULL);
zombie.c:23:        execlp("ls", "ls", "-l", (char *)NULL);
zombie.c:29:    sleep(20);
root@ubuntu-4gb-hel1-1:~/os-lab7# gcc -std=c11 -Wall -Wextra -o run run.c
root@ubuntu-4gb-hel1-1:~/os-lab7# gcc -std=c11 -Wall -Wextra -o zombie zombie.c
root@ubuntu-4gb-hel1-1:~/os-lab7# gcc -std=c11 -Wall -Wextra -o redirect redirect.c
root@ubuntu-4gb-hel1-1:~/os-lab7# gcc -std=c11 -Wall -Wextra -o pipeline pipeline.c
root@ubuntu-4gb-hel1-1:~/os-lab7# ./run
parent: my PID is 2311479
child:  my PID is 2311480, my parent is 2311479
total 116
-rw-r--r-- 1 root root   408 Oct 19 12:30 out.txt
-rwxr-xr-x 1 root root 16304 Oct 26 10:32 pipeline
-rwxr-xr-x 1 root root 16312 Oct 19 12:30 pipeline_broken
-rw-r--r-- 1 root root   759 Oct 19 12:30 pipeline_broken.c
-rw-r--r-- 1 root root   753 Oct 19 12:30 pipeline.c
-rwxr-xr-x 1 root root 16344 Oct 26 10:32 redirect
-rw-r--r-- 1 root root   622 Oct 19 12:30 redirect.c
-rwxr-xr-x 1 root root 16400 Oct 26 10:31 run
-rw-r--r-- 1 root root   710 Oct 19 12:29 run.c
-rw-r--r-- 1 root root   710 Oct 19 12:27 run-ls.c
-rwxr-xr-x 1 root root 16440 Oct 26 10:31 zombie
-rw-r--r-- 1 root root   725 Oct 19 12:29 zombie.c
parent: child 2311480 ended with status 0
root@ubuntu-4gb-hel1-1:~/os-lab7#
```

Figure 1 - Linux source verification, compilation, and fork-exec-wait result

Observation

The child reported parent PID 2311479 and was reaped with status 0. Its ls output appeared between the child and final parent messages. A successful exec does not return to the child program.

1.2 A Zombie Process

The child finished ls while its parent slept for 20 seconds before calling waitpid.
During that interval the process table displayed the child as [ls] <defunct>.
After the parent finished, the zombie entry was no longer present.

```
./zombie > /dev/null &
parent_pid=$!
sleep 2
ps -o pid,ppid,stat,cmd --ppid "$parent_pid"
sleep 20
ps -o pid,ppid,stat,cmd --ppid "$parent_pid"
wait "$parent_pid"
```

```
root@ubuntu-4gb-hel1-1:~/os-lab7# ./zombie > /dev/null &
[1] 2311905
root@ubuntu-4gb-hel1-1:~/os-lab7# parent_pid=$!
root@ubuntu-4gb-hel1-1:~/os-lab7# sleep 2
root@ubuntu-4gb-hel1-1:~/os-lab7# ps -o pid,ppid,stat,cmd --ppid "$parent_pid"
    PID    PPID STAT CMD
2311906 2311905 Z    [ls] <defunct>
root@ubuntu-4gb-hel1-1:~/os-lab7# sleep 20
[1]+  Done                    ./zombie > /dev/null
root@ubuntu-4gb-hel1-1:~/os-lab7# ps -o pid,ppid,stat,cmd --ppid "$parent_pid"
    PID    PPID STAT CMD
root@ubuntu-4gb-hel1-1:~/os-lab7# wait "$parent_pid"
root@ubuntu-4gb-hel1-1:~/os-lab7#
```

Figure 2 - Zombie child before and after the parent terminates

Observation

PID 2311906 had PPID 2311905 and state Z while its parent was alive.
The second ps query showed no child after the parent completed waitpid.

1.3 Standard Output Redirection

The child closed descriptor 1 and opened out.txt. open reused the lowest available descriptor, so ls wrote its listing into the file. The diagnostic written to stderr and the parent completion message remained visible on the terminal.

```
./redirect
head -3 out.txt
```

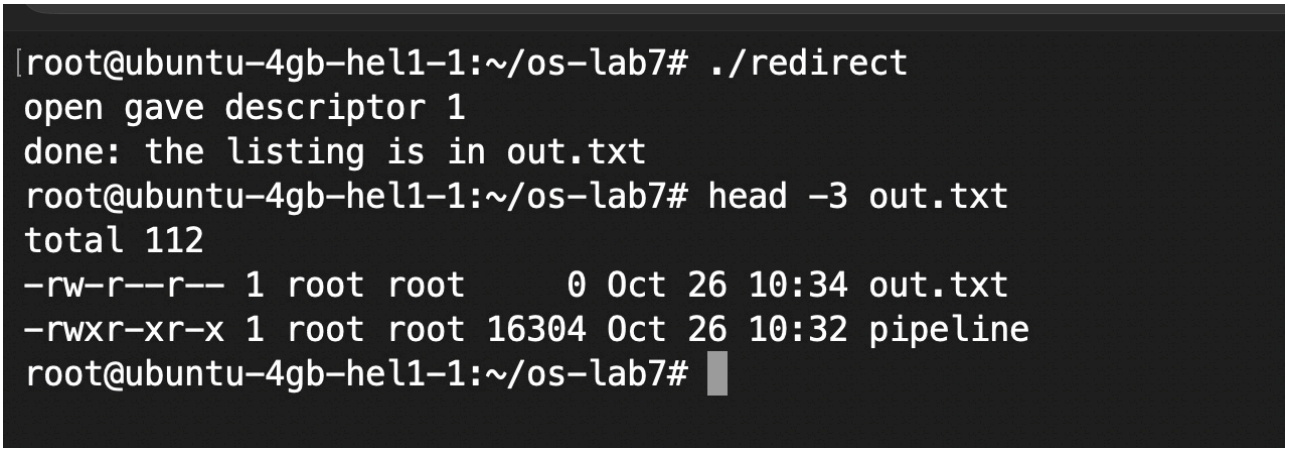


Figure 3 - Redirecting ls output through descriptor 1

Observation

open returned descriptor 1. head displayed the start of the listing stored in out.txt; the zero-byte size shown for out.txt is its size when ls listed it.

1.4 Constructing a Pipeline

One child connected stdout to the pipe write end and executed ls /etc. The other connected stdin to the read end and executed wc -l. The parent closed both ends and waited for the children.

```
./pipeline
ls /etc | wc -l
```

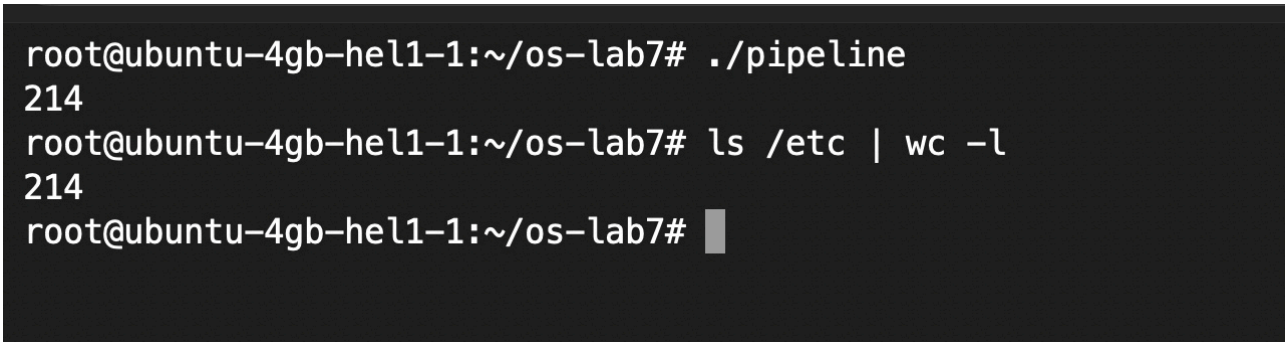


Figure 4 - Program and shell pipeline produce the same count

Observation

Both commands counted 214 entries in /etc on this machine. Closing unused pipe ends allows wc to receive EOF after ls exits and then print the count.

2 Laboratory Work No 8 The Same Calls Inside xv6

2.1 Registering User Programs

The source files user/run.c and user/pingpong.c were present in the xv6-riscv tree. The Makefile registered them as $U/_run and $U/_pingpong in UPROGS.

```
loredana@ubuntu-4gb-hel1-1:~$ cd ~/xv6-riscv
loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$ pwd
/home/loredana/xv6-riscv
loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$ ls -l user/run.c user/pingpong.c
-rw-rw-r-- 1 loredana loredana 551 Oct 19 12:39 user/pingpong.c
-rw-rw-r-- 1 loredana loredana 596 Oct 19 12:52 user/run.c
loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$ grep -n -F -e '$U/_run' -e '$U/_pingpong' Makefile
133:    $U/_run\
134:    $U/_pingpong\
loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$ grep -n -A5 -B3 'exec(' user/run.c
16-  if(pid == 0){
17-    printf("child: my pid is %d\n", getpid());
18-    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
19:    exec("echo", args);
20-    printf("exec failed\n");
21-    exit(1);
22-  }
23-
24-  int status;
loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$
```

Figure 5 - xv6 source files, UPROGS entries, and the exec call in run.c

2.2 fork exec and wait Inside xv6

The child called exec with echo and its argument vector. The parent called wait and printed the returned status after the child completed.

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$
```

Figure 6 - xv6 run executing echo

Observation

The echo program printed "hello from the child" after exec replaced run in the child. The parent reported that the child ended with status 0.

2.3 Replacing the Child with ls

The argument vector and exec target were changed from echo to ls. The output listed the xv6 file system, including run and pingpong, and the parent received status 0.

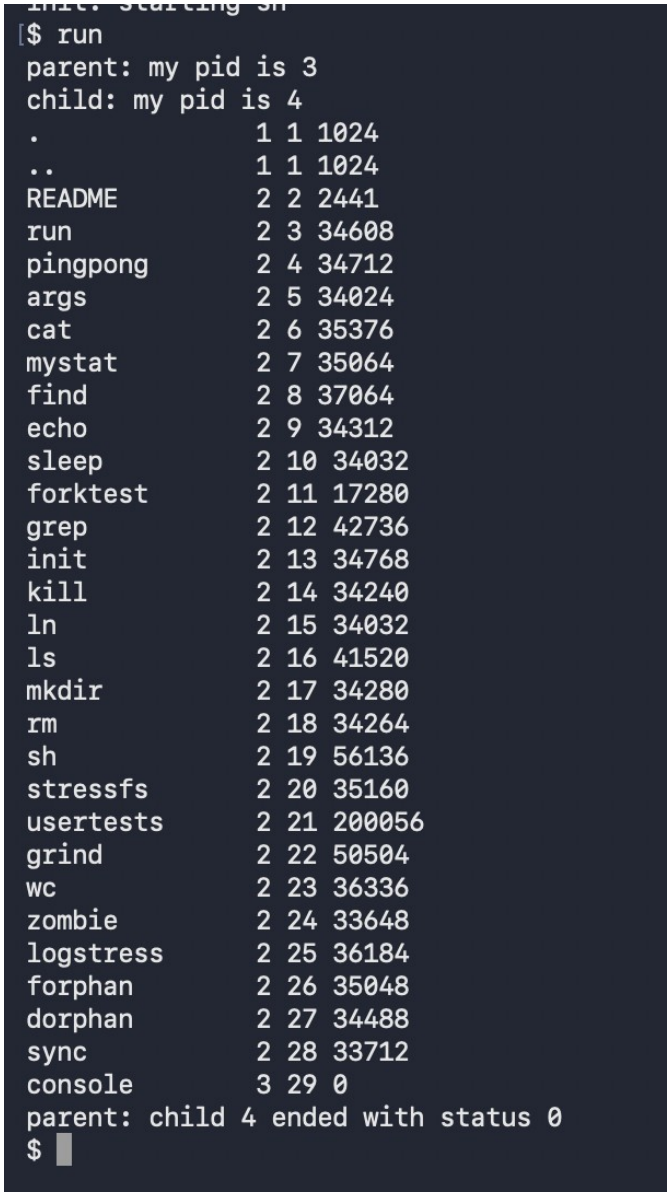


Figure 7 - xv6 run executing ls

2.4 Failed exec

The target was changed to nosuchprog. Because the executable did not exist, exec returned to the following line, which printed an error and called exit(1).

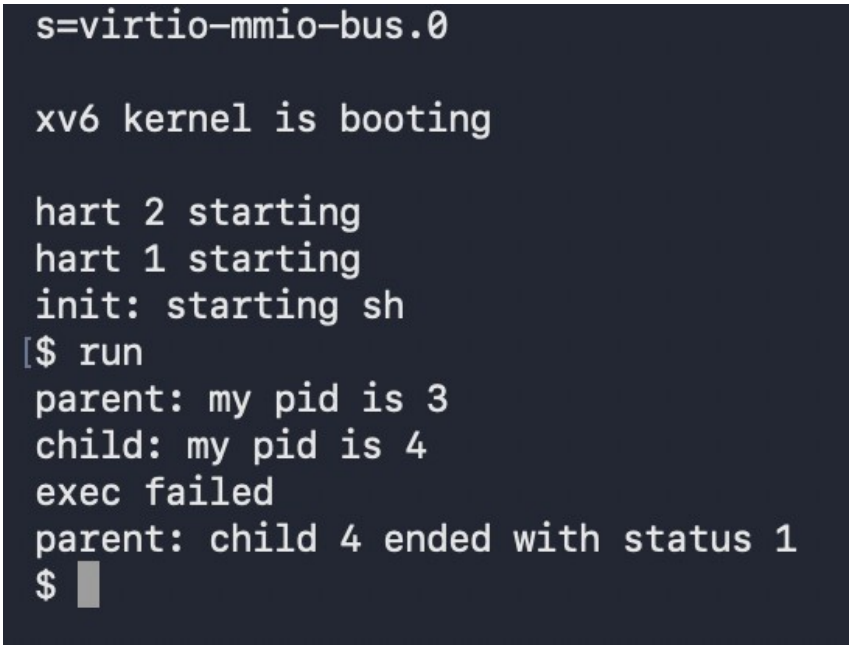


Figure 8 - xv6 failed exec and status 1

Observation

The failure status differs from Linux because the two programs explicitly selected different values: the xv6 source calls exit(1), while the Linux source calls exit(127).

2.5 Bidirectional Communication with pingpong

Two pipes were created because each pipe carried data in one direction. The parent wrote one byte to p2c, the child read it and printed the ping message, and the child returned the byte through c2p. The parent's read from c2p blocked until that reply was available.

```
run
pingpong
```

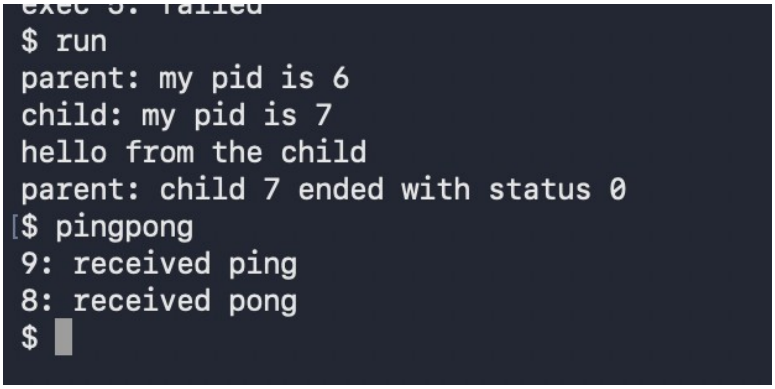


Figure 9 - Final xv6 verification of run and pingpong

Observation

Separate pipes prevent the two directions from competing for the same byte stream. The blocking read(c2p[0], &b, 1) makes the parent wait for the child's reply; wait(0) subsequently collects the terminated child. The observed order was ping first and pong second.

Appendix A Linux run.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }
    int status;
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Appendix B Linux zombie.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }
    int status;
    sleep(20);
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Appendix C Linux redirect.c

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

Appendix D Linux pipeline.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }
    if (fork() == 0) {
        close(1); dup(p[1]);
        close(p[0]); close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls"); exit(127);
    }
    if (fork() == 0) {
        close(0); dup(p[0]);
        close(p[0]); close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc"); exit(127);
    }
    close(p[0]); close(p[1]);
    wait(NULL); wait(NULL);
    return 0;
}
```

Appendix E xv6 user run.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();
  if(pid < 0){ printf("fork failed\n"); exit(1); }
  if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);
    printf("exec failed\n");
    exit(1);
  }
  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

Appendix F xv6 user pingpong.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];
  char b;
  pipe(p2c); pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);
    read(p2c[0], &b, 1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);
    close(p2c[0]); close(c2p[1]);
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);
  b = 'x';
  write(p2c[1], &b, 1);
  read(c2p[0], &b, 1);
  printf("%d: received pong\n", getpid());
  close(p2c[1]); close(c2p[0]);
  wait(0);
  exit(0);
}
```

Conclusion and Critical Analysis

The Linux and xv6 experiments show how a few system calls produce familiar shell behavior. fork created a child, exec replaced its program without creating another process, and wait or waitpid collected its termination status. The zombie experiment showed why a parent must reap a child after it exits.

The redirection experiment demonstrated that changing descriptor 1 changes where an unchanged ls program writes. The manual pipeline produced the same count as the shell pipeline; closing unused pipe ends lets the reader observe EOF.

In xv6, run repeated the fork-exec-wait pattern and pingpong used two pipes for communication in opposite directions. These experiments connect process state, descriptors, synchronization, and interprocess communication.